An Empirical Study of GPU Kernel Performance Gaps in Modern Domain-Specific Languages

Anonymous Author(s)
Anonymous Institution

***Abstract*—Modern GPU domain-specific languages (DSLs) such as Triton and TileLang deliver competitive performance relative to vendor libraries for GPU kernel programming, making them a popular choice for human developers and an increasingly common target for automated kernel code generation agents. Yet developer-written or agent-generated DSL kernels can be *functionally correct* but offer no dependable way to know whether they are *performant*. We show that a kernel can pass the prevailing evaluation benchmarks and still run hundreds of times slower than the vendor-library baseline across 22 kernels in five categories.**

The benchmarks practitioners rely on do not surface this. Prevailing benchmarks (e.g., KernelBench) gate primarily on correctness and therefore admit performance-poor kernels, while under-covering DSLs, data types, shapes, and hardware and only partially preventing reward hacking. A benchmark comprehensive enough to close this gap is combinatorially infeasible.

As a pragmatic alternative, we (i) characterize how large and structured the hidden performance gap is and why it arises, separating user-space *authoring* causes from *code-generation* and *library-maturity* causes; and (ii) distill the result into lightweight, partly baseline-independent evaluation heuristics and a small set of recurring optimization patterns that guide DSL kernel development when no ground-truth benchmark exists. We find that authoring idiom dominates the gap: a single reduction-primitive change recovers the normalization and reduction family to library parity or better (with one hardware-specific exception we diagnose), while a residual structural gap persists on convolution. We further show that a roofline-anchored comparability check flags poor kernels that correctness-only benchmarks pass.

***Index Terms*—GPU kernels, domain-specific languages, Triton, TileLang, empirical study, performance analysis**

I. Introduction

Deep learning efficiency depends on a small number of GPU kernels that dominate execution time [1], [2]. Frameworks historically delegated these kernels to vendor libraries such as cuBLAS and cuDNN [3], [4], but modern models increasingly need fused, specialized, and model-specific operators that fall outside the fixed functionality those libraries expose [5]. Writing efficient custom kernels is therefore now the bottleneck, and domain-specific languages (DSLs) such as Triton [6] and TileLang [7] have become the standard way to write them: the programmer expresses computation at the tile level and delegates memory movement, scheduling, and code generation to a compiler, in exchange for the promise of vendor-comparable performance at far lower effort. DSLs are increasingly the output of LLM-based kernel generators [9] rather than written by hand. Whether a given DSL kernel actually delivers on that promise is therefore a first-order concern.

A developer, or a generator, who produces a functionally correct DSL kernel cannot tell whether it is performant. A correct kernel may match the vendor library or run orders of magnitude slower, and nothing in the source reveals which. When a kernel is correct but slow, the developer must still localize the loss to the algorithm, the schedule, or the compiler, and existing toolchains offer little diagnostic support for that distinction [10]. This is a software-engineering problem as much as a compiler one: a high-level kernel language is useful only if its performance behavior can be evaluated and reasoned about.

The benchmarks practitioners rely on do not close this gap. The prevailing DSL and LLM-generated-kernel benchmarks, KernelBench [9] and TritonBench [11], gate on correctness and report performance as an outcome rather than enforcing it. They therefore admit performance-poor kernels: a naive but idiomatic TileLang kernel passes the KernelBench-style gate while running more than 300$\times$ slower than PyTorch, and the gate's lone reward-hacking guard never fires because it watches only for suspiciously fast kernels. These benchmarks also cover a narrow slice of the space, often one DSL, data type, shape, and GPU per task, so passing certifies little about quality. A benchmark comprehensive enough to span every operator, data type, shape, and architecture is combinatorially infeasible to build and maintain.

We study this evaluation problem through three research questions. **RQ1**: can existing benchmarks distinguish well-written DSL kernels from performance-poor ones, and where do they fall short? **RQ2**: for kernels that pass, how large and structured is the hidden gap to vendor libraries, and what causes it? **RQ3**: how can developers judge and improve DSL kernels when a comprehensive benchmark is infeasible? We answer these by evaluating Triton and TileLang against cuBLAS/cuDNN-backed PyTorch on 22 kernels across five operator classes, with Nsight Compute hardware-counter profiling on two datacenter architectures (Section III).

This paper makes the following contributions:

- **Evaluation gap (RQ1).** We show, end-to-end against the actual evaluator, that prevailing DSL and LLM-generated-kernel benchmarks admit performance-poor kernels, and we characterize their coverage and reward-hacking gaps.
- **Hidden gap and causes (RQ2).** Across 22 kernels and five operator classes on two GPU architectures, we characterize the performance gap these benchmarks miss and trace it with hardware counters to authoring, code-generation, and library-maturity causes. The dominant

TileLang normalization gap is an authoring artifact that a single reduction-primitive change removes, whereas convolution and large GEMM are structural residuals.
- **Guidance without a benchmark (RQ3).** We propose and validate two lightweight heuristics, a library-comparability screen and a baseline-independent roofline anchor, together with a small set of recurring optimization patterns that flag poor kernels and recover most of the gap.
- **Artifact.** Our kernel suite, profiling infrastructure, and analysis are publicly available at https://anonymous.4open.science/r/GPUKernelPerformance-6132/.

II. Background

A. Tile-Based GPU DSLs

A GPU executes thousands of threads grouped into *thread blocks*, each resident on a *streaming multiprocessor* with fast programmer-managed scratchpad (*shared memory*); high-performance kernels *tile* data [1] to reuse that scratchpad and stay compute-bound under the roofline model [2]. Two Python-embedded DSLs raise this tiling abstraction above hand-written CUDA, and our study targets both.

Triton [3] makes the *tile*—a statically shaped, multi-dimensional array operated on by all threads in a program instance—the unit of computation. Its MLIR-based compiler [4] performs memory coalescing, shared-memory allocation, thread swizzling, and software pipelining automatically while targeting NVIDIA PTX, and programmers tune tile configurations through `@triton.autotune`. Triton is the default lowering target for `torch.compile` since PyTorch 2.0 [5]. Its convolution support is less mature: an im2col-style lowering exists but falls substantially short of cuDNN on common workloads [6].

TileLang [7] separates the *algorithm* (what to compute) from the *schedule* (how to map computation onto GPU resources) more explicitly than Triton, letting the programmer tune memory staging, thread layout, and pipeline depth without changing the functional description. It compiles through Apache TVM with explicit hardware-intrinsic support on NVIDIA and AMD architectures. Its convolution performance relative to cuDNN has not been systematically evaluated prior to this work.

Throughout, we compare both DSLs against the vendor libraries they aim to displace: cuBLAS [8] for dense GEMM and cuDNN [9], [10] for convolution and the other deep-learning primitives, each of which dispatches the fastest implementation per problem shape via a runtime selector [11].

B. Benchmarking DSL and LLM-Generated Kernels

Custom GPU kernels are increasingly produced not by hand but by automated tools: `torch.compile` lowers operators to Triton [5], and a growing body of work uses LLMs to generate kernels directly [12], [13]. This has made kernel *benchmarks* the de-facto arbiters of kernel quality, and two dominate the DSL and LLM-generated setting.

TritonBench [14] is a collection of production-grade Triton kernels curated from high-starred open-source repositories, covering attention variants, GEMM, softmax, normalization, and fused operations. It reports both functional correctness and efficiency metrics—memory-bandwidth utilization and GPU efficiency as a fraction of theoretical peak TFLOPS—though for Triton only, on a single GPU, and for one data type and shape per operator. Our kernel suite is derived from TritonBench's GitHub channel (184 kernels from 95 repositories), supplemented with TileLang re-implementations and additional convolution configurations.

KernelBench [12] is the de-facto benchmark for LLM kernel generation: each task poses a PyTorch reference module and accepts a generated kernel—CUDA in the original, and Triton/TileLang via the same harness in our artifact—that reproduces the reference output on randomized inputs. It records a speedup over the eager baseline but *gates on correctness alone*, and its sole reward-hacking guard flags only kernels that run implausibly *fast*. Neither benchmark rejects a kernel for being slow, and neither spans more than a narrow slice of the (DSL $\times$ data type $\times$ shape $\times$ architecture) space—the evaluation gap we quantify in section V.

III. Methodology

We evaluate whether current GPU DSLs can match vendor libraries on representative deep learning operators, and, when they do not, identify the cause of the gap. To this end, we construct a kernel suite that covers the main computation patterns in modern models, compare DSL implementations against library-backed PyTorch baselines under matched configurations, and profile both end-to-end performance and low-level hardware behavior.

A. Kernel Suite

Our benchmark suite covers five operator categories that capture the dominant computation patterns in modern deep learning: matrix multiplication, attention, convolution, normalization, and element-wise or reduction operators. In total, the suite contains 22 kernels: three **GEMM** workloads (i.e., dense matrix multiplication, batched matrix multiplication, and fused linear plus activation), one **attention** workload (i.e., scaled dot-product attention / FlashAttention variants), one **convolution** workload (i.e., Conv2d with 1×1 to 7×7 filters, including depthwise and strided cases), two **normalization** workloads (i.e., LayerNorm and RMSNorm), and fifteen **element-wise or reduction** workloads including add, mul, relu, leaky_relu, softmax, log_softmax, logsumexp, swiglu, argmax, max reduction, mean reduction, cross-entropy, matrix transpose, index select, and embedding.

We derive these workloads from TritonBench [14], the TileLang example repository [7], and our own implementations for configurations not covered by either source. We exclude operators with sparse inputs or runtime-dependent output shapes, since they do not admit stable or meaningful library baselines.

a) Selection criteria.: We narrow the kernels in TritonBench [14] to our 22-kernel suite using four criteria. First, *forward-pass only*: we exclude backward and gradient kernels, whose performance is governed by different access patterns

and reduction structures. Second, *stable library baseline*: each kernel must admit a well-defined cuBLAS, cuDNN, or PyTorch reference that computes the same function, so that library efficiency is a meaningful quantity. Third, *five-category coverage*: we retain kernels that populate the five operator categories above rather than over-sampling any single class. Fourth, *canonical operators*: we exclude operators with sparse inputs or runtime-dependent output shapes, which lack a stable library baseline. The Triton implementations are drawn from TritonBench, except `layer_norm`, which follows the TorchInductor reference; the TileLang implementations are our re-implementations of the same operators, and any kernel absent from both source suites is implemented by us.

B. Baseline Construction

For each workload, we compare DSL implementations against the corresponding vendor-library path exposed through PyTorch. GEMM baselines use cuBLAS via `torch.matmul`. Convolution baselines use PyTorch `nn.Conv2d` with default NCHW memory layout under standard cuDNN algorithm selection. Normalization baselines use `F.layer_norm` and `F.rms_norm`. Element-wise and reduction baselines use standard PyTorch eager execution.

For attention, we use PyTorch scaled dot-product attention with the FlashAttention backend enabled through `enable_flash_sdp(True)`. We report these results separately because the Triton and TileLang implementations are not algorithmically identical to the PyTorch baseline.

C. Profiling Setup

We measure both end-to-end latency and hardware performance counters. Latency is used for all comparisons against library baselines, while hardware counters are used to investigate the causes of observed performance gaps.

a) End-to-end throughput.: For each workload and input configuration, we measure host-synchronized GPU execution time using `time.perf_counter()` bracketed by `torch.cuda.synchronize()` at entry and exit. Each measurement uses 10 warm-up iterations followed by 100 timed iterations, and we report the median over the timed runs. All results reflect GPU-side execution only and exclude host-side dispatch overhead. To reduce measurement noise, we run each workload in isolation without concurrent GPU activity. We pin the clocks for all timing at graphics 1215 MHz and memory 1215 MHz; the 1410 MHz maximum power-caps under the 400 W board limit, whereas 1215 MHz holds flat, yielding a run-to-run relative standard deviation of 0.0–0.9%.

b) Hardware performance counters.: To support root-cause analysis, we collect hardware counter profiles with NVIDIA Nsight Compute [15]. The RQ2 analysis is grounded in seven concrete counters: `gld_efficiency` (global-load bytes-per-sector efficiency), `ldg_per_request` (sectors per load request), `n_regs` (registers per thread), `n_spills` (register spills), `warp_stall_long_scoreboard` (memory-latency stall cycles), `warp_stall_barrier` (synchronization stall cycles), and `lts__t_sector_hit_rate` (L2 sector hit rate). Each profile is collected from a single execution, and we verify stability within 2% across five repeated runs.

c) Correctness.: Before any timing, we validate every DSL kernel against its library baseline using per-dtype numerical tolerances: `fp32` at `atol/rtol = 1e-5`, `fp16` at `1e-3`, and `bf16` at `1e-2`, with the tolerance relaxed by $2\times$ for reduction kernels to absorb order-of-summation differences. We additionally exercise an edge-case suite (NaN, Inf, denormal, and all-equal inputs) across all 22 kernels, observing 0 crashes. After applying the RQ3 mitigations, we revalidate the fixed kernels and obtain 5/5 passes, confirming the optimizations preserve numerical equivalence. We exclude FP32 GEMM from the correctness-gated comparison because its discrepancy is root-caused as silent TF32 lowering on the SM80 tensor cores rather than a kernel defect (see section VI).

D. RQ3 Optimization Methodology

To produce the optimized kernels evaluated in section VII, we employ an agentic AI kernel code generation tool [16] that, under a fixed protocol (correctness-gated iterations, fixed large-input shapes, no language switching), iteratively rewrites each flagged kernel and re-profiles it. We use the tool only as a reproducible mechanism to *apply* the root-cause-derived optimization patterns of section VI: the agent performs kernel generation and optimization, while the characterization (section V), the counter-grounded root-cause explanation (section VI), and the generalization and validation of the resulting patterns (section VII) are ours. The performance gaps we characterize and their root causes are properties of idiomatic DSL code and its compilation, independent of whether a kernel is written by hand or generated; every pattern is human-legible and hand-applicable (the dominant fix replaces a manual reduction loop with the pre-existing `T.reduce` primitive). Each optimized kernel is validated provenance-agnostically—correctness against the same per-dtype tolerances (5/5 pass), plus Nsight Compute counters and the roofline anchor—so kernel quality does not rest on trusting the tool, and we report recovered performance as a lower bound on user-space-recoverable performance.

E. Metrics

Our primary metric is **library efficiency**,

$$E_{\text{lib}} = \frac{t_{\text{lib}}}{t_{\text{DSL}}} \times 100\%,$$

where t_{lib} denotes the latency of the cuBLAS or cuDNN baseline and t_{DSL} denotes the latency of the corresponding DSL implementation under the same hardware and input configuration. A value of 100% indicates parity with the library baseline; lower values indicate that the DSL implementation is slower.

Secondary metrics: throughput ($T = 2 \cdot \text{FLOPs}/t$), hardware efficiency ($E_{\text{hw}} = T/T_{\text{peak}}$), and memory bandwidth utilization ($B = \text{bytes}/(t \cdot B_{\text{peak}})$)—are used qualitatively in root-cause analysis (RQ2) to interpret counter measurements; they are not tabulated separately.

F. Evaluation-Gap and Heuristic Measurement

To test whether existing benchmarks admit slow kernels (RQ1), we run each naive DSL kernel through a KernelBench-style evaluator (`eval_kernel_against_ref`), which overrides the candidate's input generators with the reference's, checks output equivalence on randomized inputs, and warns only when a kernel exceeds $10\times$ the reference speed; we record the pass/fail verdict and the measured slowdown against the PyTorch baseline. To compute the roofline anchor (RQ3), we model each kernel's essential work—bytes moved for memory-bound kernels (input plus output tensors) or FLOPs for compute-bound kernels—and divide the achieved work-rate by the relevant hardware peak (HBM bandwidth or FP16 Tensor-Core throughput) for the measured GPU; the peak assumed for each kernel is recorded alongside its measurement.

G. Experimental Setup

a) Hardware.: We measure on two primary datacenter architectures. The first is an NVIDIA A100-SXM4-40GB GPU (Ampere, `sm_80`) with 108 SMs, 40 GB HBM2e memory ($\sim$1.5 TB/s peak memory bandwidth), a 40 MB L2 cache, and a 400 W power cap (NVIDIA Driver 610.43.02, CUDA 12.8), on which we measure the suite magnitude (section V) and root-cause counters (section VI). The second is an NVIDIA GH200 (Grace Hopper, `sm_90`; 96 GB HBM3 at $\sim$4.0 TB/s, $\sim$989.5 TFLOP/s FP16 Tensor-Core, clocks locked at 1320/2619 MHz), on which we run the evaluation-gap demonstration (section V-B) and validate the roofline heuristic (table VII). Per-architecture magnitudes are reported where they differ.

b) Software.: Our software stack consists of PyTorch 2.8.0+cu128, Triton 3.4.0, TileLang 0.1.6.post1, bundled cuDNN, and Nsight Compute 2026.2.0. We pin all versions in a reproducibility manifest distributed with the benchmark suite. For all experiments, we run under default cuDNN flags and pre-warm the Triton auto-tuner before measurement.

IV. Research Questions

We organize the study around three research questions. The first asks whether existing benchmarks can tell a well-written DSL kernel from a performance-poor one; the second characterizes and explains the performance gap those benchmarks fail to surface; the third asks how to guide kernel development when a comprehensive benchmark is infeasible.

RQ1 (Evaluation gap). Do existing DSL and LLM-generated-kernel benchmarks distinguish well-written kernels from performance-poor ones, and along which dimensions—DSL coverage, data type, input shape, hardware, and reward-hacking prevention—do they fall short?

RQ2 (Hidden gap and its causes). For kernels that pass existing correctness-style benchmarks, how large and structured is the performance gap to vendor libraries, and which authoring, code-generation, and library-maturity factors cause it?

RQ3 (Guidance without a comprehensive benchmark). Absent a comprehensive benchmark, can lightweight evaluation heuristics—comparability with the vendor baseline plus a roofline anchor—together with recurring optimization patterns reliably flag poor kernels and guide fixes? Is there a single dominant fix, and where are DSLs genuinely limited?

Our primary measurements span two datacenter architectures, the NVIDIA A100-SXM4-40GB (Ampere, `sm_80`) and the NVIDIA GH200 (Grace Hopper, `sm_90`), spanning two GPU families.

V. The Evaluation Gap (RQ1)

This section answers RQ1: *do existing benchmarks distinguish well-written DSL kernels from performance-poor ones, and along which dimensions do they fall short?* We survey what the prevailing DSL and LLM-generated-kernel benchmarks measure (section V-A), show that a correct-but-slow kernel passes their gate unflagged (section V-B), and then quantify the hidden performance gap this gate admits across the 22-kernel suite (section V-C onward).

A. What Existing Benchmarks Measure

The two benchmarks that dominate DSL and LLM-generated GPU-kernel evaluation are KernelBench [12] and TritonBench [14], and both treat performance as a *reported outcome* rather than an *acceptance criterion*. KernelBench, the de-facto target for LLM kernel generation, accepts a candidate on *correctness alone*: its evaluator runs the kernel against a randomized-input PyTorch reference and the task passes if the outputs match within tolerance, with measured speedup recorded but never gated. Its only guard against reward hacking is a heuristic warning when a kernel runs more than $10\times$ faster than the reference—so it fires only on suspiciously *fast* kernels and is silent on the slow ones, which are the failure mode we study here.[1] TritonBench is Triton-only and, to its credit, reports a roofline-anchored GPU-efficiency metric (achieved fraction of hardware peak)—a precedent we build on in section VII—but it likewise reports rather than gates on that metric, applies no anti-cheat, and fixes one data type and shape per operator on a single GPU. Neither benchmark covers more than a narrow slice of the (DSL $\times$ data type $\times$ shape $\times$ architecture) space, and neither rejects a kernel for being slow. A kernel that "passes" therefore certifies correctness, not quality.

B. Passing Is Not Fast

To show that this gap is not hypothetical, we run idiomatic but unoptimized DSL kernels—the kind a developer or an LLM generator would plausibly produce—through the KernelBench-style correctness gate and then time them. On the GH200, the live shipped TileLang LayerNorm kernel passes the gate on all five correctness shapes yet runs $323\times$ slower than PyTorch at the large shape under locked clocks (51.5 ms vs. 0.16 ms); shipped TileLang RMSNorm passes and is $117\times$ slower. A deliberately constructed worst-case LayerNorm

[1]We exercise this evaluator directly: the KernelBench-style harness in our artifact (`eval_kernel_against_ref`) overrides the candidate's input generators with the reference's and warns on more-than-$10\times$ speedups, the same anti-cheat logic shipped upstream.

variant—fp32 internal I/O and a zero-initialized output buffer—is slower still, reaching 1293× (176.4 ms, unlocked); even a naive argmax passes and is 16.7× slower. In every case the gate returns `correct=True` and the more-than-10× reward-hacking guard never triggers, because the kernels are slow, not fast. The shipped kernels are not adversarial constructions: they are the reduction kernels shipped as idiomatic TileLang, differing from a competent implementation only in the reduction primitive (section VI). The same authoring idiom shows the same qualitative gap on the A100 suite below, where the live shipped TileLang LayerNorm is 1087× slower (table III; A100-SXM4, unlocked profile), with the suite magnitude and root-cause analysis measured on the A100 and the evaluation-gap and roofline results on the GH200; we report per-architecture magnitudes throughout and note where they differ.

C. The Hidden Gap These Benchmarks Admit

Figure 1 summarizes library efficiency (%) for all kernels in the suite, broken down by DSL (Triton, TileLang) and kernel category. The key finding is that the performance gap is not uniform: Triton is broadly competitive for element-wise and normalization kernels (88.7% of PyTorch throughput for LayerNorm and ∼95–116% for element-wise kernels; the RMSNorm outlier at 851.8% is due to kernel fusion; see table III) but trails substantially on convolution (19.3%) and large square GEMM (32.8%); TileLang achieves competitive throughput only on a narrow set of large shapes while exhibiting severe underperformance on reductions and normalizations (0.1% of PyTorch throughput on layer normalization). These figures are for the *idiomatic* kernels shipped in each DSL's examples; we show in RQ3 (section VII) that the reduction and normalization gaps are authoring artifacts—a competent `T.reduce` rewrite recovers the family to PyTorch-comparable or better (with one hardware-specific exception we diagnose in section VII-E)—whereas the convolution and large-GEMM gaps largely persist even after best-effort optimization, marking the genuine residual.

Finding: On the A100-SXM4, Triton achieves 32.8–92.9% of PyTorch/cuBLAS throughput on GEMM depending on shape; TileLang achieves 7.0–80.0% with a complementary strength profile that reorders on the GH200 (table IV). TileLang achieves 548.6% on fused linear+activation by fusing two passes into one kernel, while Triton degrades to 32.8% on the 16384^2 square matmul where its auto-tune search space is under-populated.

Table I reports latency (ms) and library efficiency for FP16 matrix multiplication across representative shapes.
Notation: 16384^2 denotes a square 16384×16384 FP16 GEMM; 64×128^2 denotes a batched GEMM of batch 64 over 128×128 matrices. FP32 TileLang GEMM is excluded; the failure is diagnosed as silent TF32 lowering in `T.gemm` (the SM80 `F32TF32TF32F32` MMA atom truncates the mantissa to 10 bits; see section VI), not a logic error.

The 32.8% efficiency on the 16384^2 matmul reflects the auto-tuning search space limitation (RC2, section VI): Triton's default configuration list for square GEMM does not cover optimal tile shapes at this scale, and register pressure from large tile configurations reduces SM occupancy. TileLang performs better on this shape (80.0%) because its explicit schedule specifies pipeline stages independently of tile size. For batched GEMM, both DSLs are competitive at the small scale (Triton 82.1%, TileLang 58.8%) but diverge at the large scale (Triton 92.9%, TileLang 7.0%), with TileLang further degraded by per-launch JIT compilation overhead. The fused linear+activation result shows TileLang achieving 548.6%— it fuses the two-pass computation into a single kernel, avoiding the memory-allocation overhead in PyTorch's eager path; Triton (72.5%) does not achieve the same fusion benefit on this shape.

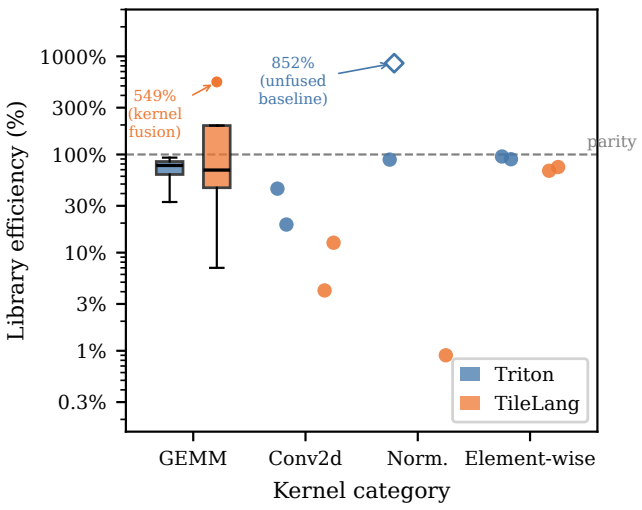

Fig. 1. Library efficiency (%) of Triton and TileLang kernels relative to cuBLAS/cuDNN on the NVIDIA A100-SXM4-40GB, grouped by kernel category (log scale). GEMM shows boxes (IQR; whiskers 1.5× IQR); all other categories show individual kernel measurements as dots ($n = 2$ per cell). Dashed line marks parity (100%). The hollow diamond at 851.8% (Triton, Norm.) marks Triton `rms_norm`: the PyTorch eager baseline does not fuse its normalization passes, making this an unfair comparison rather than a genuine DSL win; see table III. The boxplot flier at 548.6% (TileLang, GEMM) marks `fused linear+activation`, a valid DSL advantage through kernel fusion; see section V-C. Attention excluded; see section V-C.

TABLE I
GEMM-family kernels (FP16) latency (ms, lower is better), per configuration on the A100-SXM4 and GH200 (both unlocked tuned profile). E_{lib} = library efficiency relative to PyTorch/cuBLAS baseline. **Bold** marks best DSL result per row.

Configuration	PyTorch	Triton	E_{lib}	TileLang	E_{lib}
Square matmul 16384^2					
A100	34.24	104.34	32.8%	**42.80**	**80.0%**
GH200	13.71	24.34	56.3%	**22.96**	**59.7%**
Batched matmul 64×128^2					
A100	0.0787	**0.0959**	**82.1%**	0.1338	58.8%
GH200	0.0670	**0.0566**	**118.4%**	0.1059	63.3%
Batched matmul 128×2048^2					
A100	0.811	**0.873**	**92.9%**	11.55	7.0%
GH200	0.351	0.535	65.6%	**0.318**	**110.4%**
Fused linear+activation $1 \times 2048 \to 16384$					
A100	36.03	49.71	72.5%	**6.57**	**548.6%**
GH200	12.48	7.35	**169.9%**	7.66	163.0%

TABLE II

Conv2d library efficiency (E_{lib}, %) on the A100-SXM4 (small and large shapes) and GH200 (large shape; unlocked tuned profile). NCHW layout (PyTorch default), FP16, padding "same". E_{lib} relative to PyTorch/cuDNN baseline (higher is better). **Bold** marks the best DSL per cell.

	A100 $8 \times 64 \times 56^2$		A100 $32 \times 256 \times 128^2$		GH200 $32 \times 256 \times 128^2$	
Filter	Triton	TileLang	Triton	TileLang	Triton	TileLang
1×1	**44.09%**	5.28%	**28.69%**	7.15%	28.97%	**32.89%**
3×3	**44.87%**	4.12%	**19.33%**	12.62%	12.14%	**54.05%**
5×5	**30.19%**	7.58%	**13.67%**	11.16%	8.08%	**49.48%**
7×7	**20.53%**	9.05%	**9.79%**	—(OOM)	9.77%	**71.70%**
depthwise 3×3	**5.40%**	0.05%	**1.16%**	0.05%	**0.84%**	0.04%
3×3 stride 2	**46.34%**	4.17%	**17.17%**	16.77%	**13.28%**	7.24%

PyTorch/cuDNN A100 large-shape ($32 \times 256 \times 128 \times 128$) baseline latencies (ms): 1×1 0.518, 3×3 3.445, 5×5 7.716, 7×7 13.846, depthwise 0.776, stride-2 1.261. TileLang 7×7 OOMs on the A100 (40 GB) but not on the GH200, where it reaches 71.7%.

a) Attention.: Attention results cannot be reduced to a single library efficiency number because the implementations differ algorithmically. Triton's kernel implements FlashAttention-2 [17] (44.83 ms at the large configuration), which rewrites the attention computation to process the score matrix in tiles, avoiding materializing the full $n \times n$ matrix; comparing this to PyTorch's standard $O(n^2)$ path (3045.0 ms) measures an algorithmic improvement, not DSL-versus-library productivity. TileLang's kernel (5458.97 ms for the same configuration) uses the standard $O(n^2)$ formulation without tiling, making it slower than even PyTorch's unoptimized path because it does not exploit the memory hierarchy for attention. Neither result constitutes a valid DSL-versus-library efficiency measurement for the same computation; both are excluded from table IV.

Finding: Triton-written Conv2d kernels achieve 9.8–28.7% of PyTorch/cuDNN throughput across standard 1×1–7×7 filters at the large shape (up to 46.3% at the small shape); TileLang Conv2d achieves 7.1–12.6%, a deficit compounded by both per-launch JIT compilation overhead and absent vectorization of strided spatial accesses. Both DSLs collapse to $\leq$5.4% on depthwise convolution, which they lower to N per-group GEMM launches.

Table II reports library efficiency for representative Conv2d configurations across six filter shapes. Inputs follow the NCHW layout (PyTorch's default arrangement).

At the large shape Triton efficiency on the standard 1×1–7×7 filters falls monotonically from 28.7% (1×1) through 19.3% (3×3), 13.7% (5×5), to 9.8% (7×7); the gap widens precisely on the Winograd-ineligible larger filters (section VI). TileLang holds a narrow 7.1–12.6% band over the same filters. Depthwise convolution, which both DSLs lower to N per-group GEMM launches, collapses to $\leq$5.4% for Triton and to $\sim$0.05% for TileLang. Nsight Compute shows the conv gap is occupancy- and coalescing-bound (RC1 + RC2), not a register-spill problem: `n_spills = 0` at every filter size for both DSLs. On the GH200, Triton convolution stays comparably weak (12.1% at 3×3, $\leq$9.8% at the larger filters), but TileLang convolution is far stronger than on the A100 (32.9–71.7% across 1×1–7×7; table II), so the TileLang convolution collapse is largely an Ampere-specific scheduling artifact rather than intrinsic to the DSL—the persistent residual is the *Triton* convolution gap, which holds on both architectures. The root causes of these gaps are analyzed in section VI.

TABLE III

Normalization latency (lower is better). E_{lib} relative to PyTorch eager baseline.

Kernel	Shape	PyTorch(ms)	Triton(ms)	E_{lib}	TileLang(ms)	E_{lib}
LayerNorm	8192^2 BF16, A100	0.335	**0.378**	**88.7%**	364.09	0.1%
LayerNorm	8192^2 BF16, GH200	0.140	**0.188**	**74.8%**	46.13	0.3%
RMSNorm	8192^2 FP16, A100	2.516	**0.295**	**851.8%**	294.27	0.9%
RMSNorm	8192^2 FP16, GH200	1.276	**0.118**	**1081%**	38.87	3.3%

A100 rows are the unlocked tuned profile; locked-clock re-timing (graphics 1215 MHz / memory 1215 MHz) over 100 runs shows run-to-run relative std-dev 0.0–0.9% on near-parity kernels, so the gaps are statistically significant. GH200 rows are the unlocked tuned profile; TileLang LayerNorm there is 0.3% (329×), matching the locked-clock 323× in section V-B within run-to-run noise.

Finding: Triton normalization kernels match or substantially outperform PyTorch's eager paths; TileLang normalization collapses to less than 1% of PyTorch throughput at the tested size (8192×8192), with LayerNorm 1087× slower.

Table III reports latency for layer normalization and RMS normalization. Triton's `layer_norm` achieves 88.7% of PyTorch throughput at the large shape (8192×8192, BF16). Triton's `rms_norm` achieves 851.8% efficiency because PyTorch's eager `rms_norm` path does not fuse the variance-reduction and normalization passes, whereas the Triton kernel issues a single fused pass. TileLang normalization is catastrophically slow: 0.1% for LayerNorm (364.09 ms vs. PyTorch's 0.335 ms, a 1087× slowdown) and 0.9% for RMSNorm (294.27 ms vs. 2.516 ms). As analyzed in section VI, this reflects a combination of per-launch JIT overhead, absent vectorized memory loads, and serialized memory-latency exposure under `T.serial` reduction loops (`warp_stall_long_scoreboard` dominates; `warp_stall_barrier` $\approx$ 0).

Finding: Hardware-agnostic heuristic tuning shows negligible benefit for Triton in the tested configuration — default and tuned configurations produce nearly identical latencies for all kernels. The convolution gap is essentially unchanged (19.3%). TileLang shows no measurable benefit from tuning across any kernel.

We re-evaluated all 22 kernels with heuristically tuned configurations for both DSLs; every kernel returned identical efficiency to its default configuration (Δ = 0pp across all rows). This indicates that either the default configurations are already optimal for this hardware, or the tuner's search space does not cover configurations that would improve performance on the NVIDIA A100-SXM4-40GB. The convolution gap (19.3%) is unchanged by tuning, consistent with RC1–RC3 being structural compiler limitations rather than search-space coverage problems (section VI).

The apparent tension between this $\Delta \approx$ 0pp result and the matmul speedup reported in section VII-D is a search-space distinction, not a shape distinction. Here, "heuristic

TABLE IV
Median library efficiency (%) per kernel category and DSL on the A100-SXM4 and GH200 (both unlocked tuned profile), computed over large-size benchmark configurations. Attention excluded from Overall; see text.

	A100-SXM4		GH200	
Kernel category	Triton	TileLang	Triton	TileLang
GEMM	32.8–92.9%	7.0–80.0%	56–170%	60–163%
Attention	*see text — baselines non-equivalent* ‡			
Convolution	19.3%	12.6%	12.1%	54.1%
Normalization	88.7%‡	0.1–0.9%‡	74.8%	0.3–3.3%
Element-wise	∼95–116%	∼67–74%	∼66–98%	∼62–68%
Overall (excl. attn.)	**∼73%**§	**∼29%**§	**∼98%**	**∼67%**

TileLang LayerNorm (large shape) is 0.1% (1087× slower than PyTorch) due to serialized memory-latency exposure under `T.serial` (`warp_stall_long_scoreboard` dominates) and absent vectorized loads; see section VI. RMSNorm Triton efficiency is 851.8% because the PyTorch eager path does not fuse normalization passes.

§ Overall figures are medians excluding attention; the Triton geomean (∼126%) is inflated by fusion and eager-baseline outliers (`rms_norm`, `leaky_relu`, `cross_entropy`, `linear+act`), so we report the more representative median. Splitting the baseline into vendor cuBLAS/cuDNN, `torch.compile(max-autotune)` fused, and eager PyTorch (`fused_baselines`) confirms the convolution gap persists after fusion.

tuning" refers to the 12-config block-tile grid (varying only block-$M/N/K$); on the 16384^2 square matmul this grid yields $\Delta \approx 0$pp (Triton 32.5% untuned vs. 32.5% tuned). The expanded search in section VII, which additionally tunes `GROUP_SIZE_M`, `num_warps`, and `num_stages`, recovers Triton from 32.5% to 81.8% at 16384^2 (a $2.52\times$ latency improvement) and from 56.4% to 76.5% at 4096^2 ($1.36\times$); the optimal configuration is simply absent from the default block-tile grid (RC2a).

Table IV summarizes median library efficiency per category and DSL.

The most striking asymmetry is TileLang's normalization collapse: a $1087\times$ slowdown on large LayerNorm (section VI) attributable to three compounding deficiencies (RC0), and it persists across architectures ($329\times$ on the GH200, table III). On the A100, Triton is broadly competitive with PyTorch for element-wise (∼95–116%) and normalization (88.7%) kernels but exhibits a persistent gap on convolution (19.3%) and large square GEMM (32.8%). TileLang on the A100 shows competitive throughput only on fused linear+activation (548.6%) and large square matmul (80.0%); all other A100 TileLang results trail PyTorch substantially, with element-wise operations at ∼67–74% and batched GEMM at 7.0%. The GH200 reorders this strength profile (table IV): TileLang is markedly more competitive there—batched GEMM 110%, convolution 54%, square matmul 60%, overall median ∼67% versus ∼29% on the A100—so the two gaps that survive on *both* architectures are the TileLang normalization collapse and the Triton convolution residual ($\leq$13% at 3×3), not a fixed per-DSL ranking. This hardware-dependence is itself the point: a DSL kernel's competitiveness cannot be read off one GPU.

Table V reports per-kernel library efficiency for the 15 element-wise and reduction kernels individually at the large input size. Entries above 100% (marked †) reflect kernel fusion or an unfused/eager PyTorch baseline rather than genuine speedups over a vendor library, and should not be read as pure DSL wins.

TABLE V
Per-kernel library efficiency (E_{lib}, %) for the 15 element-wise and reduction kernels at the large input size, NVIDIA A100-SXM4-40GB. † marks entries >100%, which reflect kernel fusion or an unfused/eager PyTorch baseline (not pure DSL speedups over a vendor library).

Kernel	Triton	TileLang
`add`	89.5%	74.5%
`mul`	69.7%	67.6%
`relu`	95.4%	68.3%
`leaky_relu`	427.9%†	1241.7%†
`swiglu`	267.1%†	104.2%†
`softmax`	116.0%†	19.0%
`log_softmax`	44.9%	17.0%
`logsumexp`	475.7%†	57.6%
`argmax`	4.7%	5.2%
`max_reduction`	12.7%	4.7%
`mean_reduction`	72.5%	7.2%
`cross_entropy`	1132.2%†	111.0%†
`embedding`	441.9%†	70.1%
`index_select`	60.5%	55.2%
`matrix_transpose`	340.1%†	107.4%†

VI. Root Causes of the Hidden Gap (RQ2)

The kernels in section V pass existing correctness benchmarks (section V-A) yet lag the vendor library by the margins reported there. This section identifies the causes of that hidden gap, separated by where the fix belongs: user-space *authoring* (RC0a), compiler *code generation* (RC0b, RC1, RC2a), and *library maturity*—the tuning and algorithm selection a mature vendor library provides (RC2b, RC4). This separation is what makes the gap actionable: authoring causes a developer can fix today, code-generation causes require compiler work, and library-maturity causes bound what any DSL kernel can currently reach.

The following root causes are motivated by latency measurements (section V) and community reports [6].

a) RC0: TileLang Compiler-Level Deficiencies.: Two mechanisms contribute to RC0, which we separate by where the fix lives: RC0a is a user-space kernel-authoring choice, while RC0b is a compiler-codegen deficiency.

b) RC0a: TileLang Reduction Authoring (`T.serial` → `T.reduce`).: First, the original TileLang normalization kernels implement the reduction over the normalized dimension using `T.serial` loops—TileLang's sequential for-loop construct—iterating over each partial sum one at a time. For LayerNorm, which accumulates 8 bfloat16 partial statistics per thread (sum and sum-of-squares for the mean and variance passes), this means the reduction is computed in 8 sequential steps with no inter-thread parallelism. The native alternative, `T.reduce`, lowers to a parallel butterfly reduction via `tl::AllReduce`. A *butterfly reduction* is a logarithmic communication pattern in which threads exchange partial results in $\log_2 N$ rounds (here $\log_2(256/32)$ rounds over 256 threads grouped into warps of 32). This algorithmic

structure, however, is not where the cost lies: on the A100, the performance gain from `T.reduce` comes from better memory-latency hiding—`warp_stall_long_scoreboard` drops from 59.56 cycles to ≈0—not from removing barriers, since `warp_stall_barrier` is already ≈0 in both the `T.serial` and `T.reduce` kernels. The GH200 `ncu` counters confirm the same signature on `sm_90`: `warp_stall_barrier`=0 with `warp_stall_long_scoreboard`=49.95 dominating, alongside a 34 GB local-store register spill (RC3). Replacing the `T.serial` loop with a single `T.reduce` call thus removes the dominant latency source: on the A100, LayerNorm drops from 364 ms to 0.270 ms (1347× improvement, section VII-B).

c) RC0b: Absent Vectorized Loads (Compiler Codegen).: Second, TileLang's compiled normalization kernels do not issue 128-bit vectorized loads (`LDG.128`), instead fetching 16-bit bfloat16 elements individually. For a tensor of shape 8192×8192, this produces an excessive number of discrete memory transactions that bottlenecks the memory controller and prevents the L2 cache from streaming coalesced data to the SMs. On the A100, NCU confirms this scalar-granularity access for TileLang LayerNorm: a load efficiency of 50% bytes/sector at one sector per request, leaving DRAM throughput at 59.2%. Unlike RC0a, this deficiency cannot be addressed in user space and requires a compiler-codegen fix.

d) TileLang float32 lowering to TF32.: A third deficiency surfaced as a correctness failure under float32 precision but is rooted in silent TF32 lowering rather than a logic error. On the A100, TileLang's `T.gemm` kernel fails an exact float32 check (atol 10^{-5}) on 31.8% of output elements, with a maximum absolute difference of 0.185 at the tested shape $4096 \times 2048 \times 1024$. This error is in the same magnitude class as cuBLAS's own TF32 path, and a manual FMA float32 kernel passes the same 10^{-5} check—isolating the fault to `T.gemm`. The mechanism is that TileLang exposes no user-space TF32 knob: float32 `T.gemm` lowers to the SM80 `F32TF32TF32F32` MMA atom, whose 10-bit input mantissa (vs. FP32's 23-bit) truncates precision. The behavior is reproducible across four isolation arms in `exp_fp32_gemm.py`. For this reason, all TileLang GEMM measurements in this paper use FP16; float32 TileLang GEMM results are excluded. This is a precision-lowering artifact, not a performance gap, but it has implications for training workloads that require true FP32 accumulation precision.

Finding RC0: TileLang's compiled kernels exhibit two structural performance deficiencies: (RC0a) `T.serial` reductions serialize memory-latency hiding rather than synchronization (`warp_stall_long_scoreboard` dominates; `warp_stall_barrier` ≈0), correctable in user space via `T.reduce`; and (RC0b) absent 128-bit vectorized memory loads for normalization, which require a compiler-codegen fix. These compound the per-kernel performance gaps measured in RQ1. A third deficiency—silent TF32 lowering of `T.gemm` under FP32—is correctness-affecting but orthogonal to performance.

e) RC1: Absent Vectorization of Strided Memory Accesses.: NVIDIA Ampere's memory subsystem achieves peak bandwidth only when global memory loads are issued as 128-bit vector instructions (`LDG.128`). On Hopper, the Tensor Memory Accelerator (TMA) similarly requires aligned, contiguous access descriptors. For GEMM kernels, Triton's compiler successfully emits vectorized loads because the data layout is row-major and tile boundaries are aligned to 16-byte boundaries.

Convolution complicates this in two ways. First, each output element requires gathering input values from a $(K_H \times K_W)$-window strided by `(stride_h, stride_w)` in the spatial dimensions. The resulting access pattern is a strided gather across the innermost dimension when the input is stored in NCHW layout (PyTorch's default, as benchmarked), breaking the alignment requirements for `LDG.128`. Second, for filter sizes $K_H \times K_W > 1$, the compiler must prove at compile time that successive iterations of the spatial reduction loop produce aligned addresses; this proof is not discharged by Triton's current alias analysis, so it conservatively emits scalar 32-bit loads.

The consequence is a cascade noted in the community [6]: un-vectorized accesses prevent `cp.async` from firing, because CUDA's asynchronous copy instructions require the source address to produce a 128-bit aligned transfer. Without `cp.async`, the software pipeline cannot overlap data movement with computation, and the kernel stalls on global memory latency at every tile boundary. On the A100's HBM2e interface (≈1.5 TB/s peak), scalar 32-bit loads achieve only a small fraction of peak bandwidth; `LDG.128` vectorized loads are required to saturate the memory bus.

Finding RC1: Triton-generated convolution code fails to issue vectorized (128-bit) load instructions, preventing asynchronous copy (`cp.async`) from being applied and collapsing the software pipeline.

NCU confirms the absent vectorization on the A100: Triton conv2d achieves a load efficiency of only 12.55% bytes/sector at 16.4 sectors per request, versus the `cp.async`-staged matmul path.[2]

f) RC2a: Configurations Absent From the Default Grid.: Triton's auto-tuner sweeps a programmer-specified list of configurations $\{(\texttt{BLOCK\_M}, \texttt{BLOCK\_N}, \texttt{BLOCK\_K}, \texttt{num\_stages}, \texttt{num\_warps})\}$. For GEMM, this 5-dimensional space is well-understood: large square tiles (128×128) with 4–8 pipeline stages are optimal across most A100 problem shapes [3]. The reference Triton GEMM tutorial ships a configuration list that achieves near-cuBLAS performance for common shapes, but omits the L2-swizzle and large-shape configurations needed at scale.

This is evidenced by the large-matmul result in section V-C: Triton achieves only 32.8% of cuBLAS throughput on a 16384^2 FP16 matrix multiplication (104.34 ms vs. PyTorch's 34.24 ms), a shape not covered by standard tutorial configuration lists,

[2]Triton matmul and cuBLAS read 0% on this load counter because their `cp.async`/`LDGSTS` staging bypasses the `LDG` instruction the counter tracks.

compared to 92.9% on a 128×2048^2 batched GEMM where the auto-tuner has well-populated configurations. The convolution gap (19.3% for the $32 \times 256 \times 128 \times 128$, 3×3 configuration) follows the same pattern at an even larger scale. Expanding the search beyond the default 12-config block-tile grid (sweeping `GROUP_SIZE_M`, `num_warps`, and `num_stages`) recovers Triton from 32.5% to 81.8% of cuBLAS at 16384^2 (a $2.52\times$ gain) and from 56.4% to 76.5% at 4096^2, confirming that the missing configurations—not an exhausted search budget—bound default performance (section VII).

g) RC2b: L2 Cache Residency.: For the 16384^2 matmul, the combined A, B, C footprint ($\approx$1.6 GB) vastly exceeds the A100's 40 MB L2 cache; cuBLAS employs hardware-specific cache-blocking (via `cuBLASLt`'s plan selector) while Triton's generic `tl.dot` compilation lacks equivalent heuristics, producing the $\approx3\times$ latency penalty observed (34.24 ms vs. 104.34 ms). On the A100, NCU confirms the residency divide: Triton's 16384^2 matmul reaches only a 49.4% L2 hit rate and is DRAM-bound at 85.9% DRAM throughput, whereas cuBLAS holds an 80.5% L2 hit rate at 28.8% DRAM throughput.

Convolution adds filter-size degrees of freedom: 1×1 convolutions behave like GEMM (large tiles preferred), while 3×3 require smaller K-dimension tiles to keep shared memory within 48 KB per SM. Default configuration lists do not cover 5×5 and 7×7 filters, leaving substantial performance on the table. TileLang additionally requires `num_stages` to be declared ahead of time; for large filters the optimal value depends on register pressure from spatial accumulation—a dependency current scheduling heuristics do not model.

Finding RC2: The static tile-configuration search space in Triton's `@triton.autotune` omits configurations needed at scale (RC2a: L2-swizzle and large-shape tiles absent from the default grid), and Triton's generic compilation lacks the L2 cache-blocking heuristics cuBLAS applies once the working set exceeds L2 (RC2b).

h) RC3: TileLang LayerNorm Register Spill (Not a Convolution Problem).: Each streaming multiprocessor on the A100 exposes a 65,536 32-bit register file, so a block of 128 threads at 64 registers per thread occupies it fully, preventing a second resident block and eliminating pipeline interleaving. Contrary to the submitted-PDF framing, NCU shows convolution does not hit this wall: Triton conv2d reports `n_spills`$=0$ at every tested filter (1×1 through 7×7), even though register usage rises with K^2 (up to 224 regs/thread at large shapes). The spill is instead TileLang-LayerNorm-specific: the large LayerNorm kernel uses 254 registers per thread, drops to 12.46% achieved occupancy, and spills 206 GB of local-load plus 137 GB of local-store traffic. This register pressure—not any convolution effect—is what collapses occupancy below the level required for latency hiding.

Finding RC3: The TileLang LayerNorm kernel exceeds the per-thread register budget (254 regs/thread), spilling to local memory (206 GB local-load + 137 GB local-store) and reducing achieved occupancy to 12.46%; convolution kernels, by contrast, do not spill (`n_spills`$=0$ at all filters).

i) RC4: Absence of Algorithm-Level Diversity.: cuDNN's runtime selector chooses Winograd for 3×3 filters when arithmetic reduction is the bottleneck. Winograd convolution reduces the multiply-accumulate count by a factor of approximately $2.25\times$ for 3×3 (Winograd $F(2, 3)$) filters, at the cost of additional data transformation passes. Neither Triton nor TileLang expose the Winograd transformation as a schedulable primitive, so they cannot exploit this arithmetic reduction. Isolating this effect on the A100, however, bounds its size: a deterministic A/B comparison of cuDNN on 3×3 stride-1 convolution differs by only 0.06% (a ratio of 0.9994), so absent Winograd selection accounts for at most $\approx$2–3% of the gap. Moreover, the measured Triton-vs-cuDNN gap *widens* on Winograd-ineligible filters (3×3 s1 19.25%, 3×3 s2 16.94%, 5×5 13.5%, 7×7 9.81%), indicating that the residual is general cuDNN implicit-GEMM tuning rather than algorithm selection.

Finding RC4: Absent Winograd selection accounts for at most $\approx$2–3% of the residual 3×3 convolution gap on the A100; the remainder reflects cuDNN's general implicit-GEMM tuning, and the gap widens on Winograd-ineligible filters.

A. Summary of Root Causes

Table VI maps each root cause to the kernel category it primarily affects, its estimated contribution to the throughput gap, and the Nsight Compute counter most diagnostic for its detection.

VII. Guidance Without a Comprehensive Benchmark (RQ3)

This section answers RQ3: *absent a comprehensive benchmark, can lightweight evaluation heuristics and a small set of recurring optimization patterns reliably flag performance-poor DSL kernels and guide their repair?* RQ1 (section V) showed that a comprehensive performance benchmark is infeasible and that correctness gates admit slow kernels; RQ2 (section VI) explained why the gaps arise. We now distill that evidence into developer-facing guidance: a two-part *evaluation heuristic* for deciding whether a kernel is well written (section VII-A), and the *optimization patterns* that repair the gaps it flags (section VII-B onward), validated by optimizing kernels across the suite. The optimization campaigns (section III-D) were run on a separate development GPU; the optimized kernels they produced are re-timed throughout this section on the A100-SXM4 and GH200 evaluation hardware. The central result is a clean dichotomy: most of the dramatic gaps are *authoring artifacts* that one dominant pattern fully repairs, while a minority are *genuine residuals* that survive best-effort optimization—and the heuristic tells the two apart without a ground-truth benchmark. We position the user-space fixes (RC0a, RC2a, the matmul L2-swizzle) as lower bounds on automatable recovery; RC0b and the broader RC2b search space require compiler support.

TABLE VI

Root-cause summary. Measured contribution is the latency speedup recovered by the corresponding mitigation (section VII); "—" indicates no direct mitigation experiment conducted. RC0a, RC0b, and RC3 are TileLang-specific; RC1, RC2, and RC4 affect both DSLs. Per-RC counter values are consolidated in the A100 and GH200 `ncu_summary.csv` files, which agree on the RC signatures.

Root cause	Affected kernels	Contribution	Diagnostic counter
RC0a: `T.serial` instead of `T.reduce`	All TileLang reductions, norm	1347× (LayerNorm, A100)	`warp_stall_long_scoreboard` (barrier ≈0)
RC0b: Absent vectorized loads / dtype cast	TileLang norm	—	`l1tex` bytes/sector
RC1: Strided access, scalar LD	Conv2d ($K > 1$), Triton	2.2× with RC2	Load bytes/sector eff. (12.55%)
RC2a: Auto-tuning mismatch	Matmul, Conv2d, Depthwise	1.37× (Matmul)	TFLOPS vs. swept configs
RC2b: L2 cache residency	Matmul $\geq 16384^2$	—	L2 hit rate, DRAM throughput
RC3: TileLang LayerNorm spill	TileLang LayerNorm (large)	—	`local_op` spill / occupancy (A100: 51.5 GB ld, 254 regs, 12% occ)
RC4: No Winograd	Conv2d 3×3	≈2–3%	SM utilization delta

A. Evaluation Heuristics: Is This Kernel Well Written?

We propose two complementary checks a developer can apply without a curated benchmark.

a) A comparability screen (pragmatic, baseline-relative).: A well-written DSL kernel should (i) come within a small factor of the vendor/PyTorch baseline on representative shapes, (ii) be at least at parity—ideally faster—at large input sizes where launch and framework overheads amortize, and (iii) show no catastrophic per-shape collapse. This screen is cheap and catches gross failures: the idiomatic TileLang LayerNorm that runs more than $300\times$ slower (section V-B) fails it immediately. Its limitation is circularity—PyTorch is simultaneously the baseline and the de-facto definition of "good," so the screen cannot certify a kernel that merely matches an already-slow baseline, and it can be fooled into crediting a kernel that beats an *unfused* eager path for the wrong reason (the RMSNorm 851% "win" in table III is a fusion artifact, not headroom). It is therefore a screen, not a certificate.

b) A roofline anchor (baseline-independent).: To break that circularity we anchor quality to the hardware rather than to PyTorch: for a kernel whose essential work is W (bytes moved if memory-bound, FLOPs if compute-bound) and whose optimized time is t, the achieved roofline fraction is $\rho = W/(t \cdot P)$, where P is the relevant hardware peak [2]. TritonBench reports an analogous achieved-peak metric [14]; we use it not as a leaderboard score but as a *decision signal*—a kernel near the memory or compute roof is well written *regardless* of what the library does. Table VII reports ρ for the optimized kernels on the A100-SXM4 (primary) and the GH200, alongside their PyTorch-relative efficiency E_{lib} and the *cliff* (naive-to-optimized speedup) that measures how much authoring headroom each kernel hid; the two architectures agree on every authoring-artifact vs. structural-residual classification (the GH200 ρ values cited below are matched within ± 0.1 on the A100). The anchor does two things the comparability screen cannot. First, it confirms the genuine residuals are genuinely far from the hardware: optimized Triton Conv2d reaches only $\rho = 0.16$ and index-returning argmax only $\rho = 0.10$, so their shortfall is real headroom, not merely a fast baseline. Second, it de-inflates baseline-relative outliers: the RMSNorm kernel that looks like a $13\times$ "win" over PyTorch sits at $\rho = 0.69$—well written, but not $13\times$ beyond what the hardware allows. The recovered normalization and reduction family lands at $\rho = 0.41$–0.87, while the residual convolution and argmax kernels sit at $\rho \leq 0.28$ even after best-effort optimization; the cliff column shows these are not the same axis—LayerNorm hid a $1541\times$ authoring artifact that the dominant pattern fully recovers, whereas Conv2d's $8.2\times$ cliff still leaves it at $\rho = 0.28$, a structural ceiling the rewrite cannot lift. We present the two checks together because neither suffices alone, and they disagree exactly in a judgment band near $\rho \approx 0.5$: Softmax and Matmul both achieve $\rho = 0.47$, yet Softmax is at parity ($E_{\text{lib}} = 0.87$) while Matmul trails cuBLAS ($E_{\text{lib}} = 0.68$)—only the two checks together make the call.

B. Normalization Kernels: Correcting RC0

a) Finding: Correcting RC0 with two optimizations (each necessary, neither sufficient alone) brings TileLang LayerNorm to 124% *and RMSNorm to* 990% *of PyTorch throughput on the A100-SXM4— a* $1347\times$ *and* $1157\times$ *reduction over the unoptimized TileLang baseline (table III): (1) replacing* `T.serial` *reduction loops with the native* `T.reduce` *primitive (the dominant step); (2) switching to native bfloat16/float16 I/O and eliminating intermediate* `.float()` *precision casts. These results confirm RC0 as the primary cause of TileLang's normalization deficit and as fully correctable in user space.:* Here one iteration denotes a single kernel-source edit plus one benchmark run with a correctness check, logged and committed in `ITERATIONS.md`; failed and regressing attempts count. The LayerNorm `ITERATIONS.md` records 18 such iterations and the RMSNorm log records one.

Step 1: Replace `T.serial` *with* `T.reduce`*:* Replacing the manual `T.serial` loop with a single `T.reduce` call is the dominant fix. Subsequent iterations explored thread count, tiling, and two-pass formulations, confirming the loop structure—not thread configuration—was the bottleneck.

Step 2: Native bfloat16 I/O: Switching to native bfloat16 tensors and removing intermediate `.float()` casts brings the kernel close to the memory-bandwidth limit: the theoretical minimum for a 8192×8192 bfloat16 tensor (read + write $\approx$ 268 MB) at the A100's ≈1.5 TB/s is $\approx$ 0.18 ms; the

TABLE VII

Evaluation heuristics on the A100-SXM4 (sm_80, primary) and GH200 (sm_90): optimized-kernel roofline fraction ρ (achieved/hardware peak; *mem*=HBM bandwidth, *comp*=FP16 Tensor-Core), PyTorch-relative efficiency E_{lib}, and the *cliff* (naive/optimized speedup). A high recovered ρ marks an *authoring artifact*; a low ρ that survives optimization marks a *structural residual*; the two architectures agree on every classification. Source: `experiments/results/NVIDIA_{A100-SXM4-40GB, GH200_480GB}/cliff_roofline.csv`.

Kernel (DSL)	Bound	Arch	ρ	E_{lib}	Cliff
Recovered family (authoring artifacts)					
LayerNorm (TL)	mem	A100	0.67	1.25	380×
		GH200	0.59	1.20	1541×
RMSNorm (TL)	mem	A100	0.70	10.2	325×
		GH200	0.69	13.0	1520×
MeanReduction (TL)	mem	A100	0.89	1.04	13.7×
		GH200	0.87	0.97	13.9×
BatchedMatmul (TL)	mem	A100	0.82	0.94	23.9×
		GH200	0.86	1.11	20.2×
MaxReduction (Tr)	mem	A100	0.69	1.13	8.8×
		GH200	0.59	1.20	6.4×
MaxReduction (TL)	mem	A100	0.42	0.68	12.8×
		GH200	0.41	0.84	16.2×
LogSoftmax (TL)	mem	A100	0.55§	0.71§	—§
		GH200	0.58	0.90	5.2×
Softmax (TL)	mem	A100	0.55§	0.86§	—§
		GH200	0.47	0.87	4.0×‡
Structural residuals (survive best-effort optimization)					
Matmul (Tr)	comp	A100	0.56	0.78	1.1×
		GH200	0.47	0.68	1.3×
Conv2d (TL)	comp	A100	0.34	0.60	6.4×
		GH200	0.28	0.63	8.2×
Conv2d (Tr)	comp	A100	0.24	0.42	1.5×
		GH200	0.16	0.34	2.5×
Argmax (TL)	mem	A100	0.15	0.27	4.5×
		GH200	0.10	0.22	3.8×

TL = TileLang, Tr = Triton. ‡ The naive Softmax falls back to `torch.softmax` at this large shape, so its cliff is not a pure TileLang authoring comparison; the optimized ρ and E_{lib} are unaffected. § On the A100 stack (TileLang 0.1.6.post1) the in-tree TileLang Softmax/LogSoftmax variant's 64 KB full-row shared-memory cache collapses occupancy (ρ=0.006/0.028 in `cliff_roofline.csv`); we report the `opt_kernels` streaming variant, which reaches parity on both architectures ($\rho = W/(t\,P)$ from its measured time, E_{lib} from `mitigation_retime.csv`); its cliff is omitted as the in-tree comparison does not transfer. `logsumexp` is omitted from the recovered family: its optimized kernel register-spills on `sm_90` (GH200; $\rho < 0.02$, $E_{\text{lib}} = 1.0\%$) but *not* on `sm_80` (A100: 0 spill, 93 regs, $E_{\text{lib}} = 582\%$, `ncu_summary.csv`)—an architecture-specific RC3-class residual discussed in section VII-E and table X.

re-timed kernel reaches 0.270 ms. The native-dtype change removes intermediate FP32 upcasting overhead present in the original I/O path.

This result does not represent a new optimization technique: `T.reduce` already existed in the API. The finding is that RC0 fully explains the normalization anomaly and is mechanically correctable without any algorithmic change.

TABLE VIII

Normalization kernel latency (ms, lower is better) before and after RC0 correction, re-timed on the A100-SXM4 (locked clocks). E_{lib} = library efficiency after fix, relative to PyTorch. *Before* is the unoptimized TileLang kernel (cf. table III); *After* is the agent-optimized kernel. The unoptimized kernels are memory-latency-bound (RC0) and hence clock-invariant, so *Before* matches the unlocked profile in table III to within noise. Input shape: 8192×8192.

Kernel	PyTorch	Before	After	E_{lib}
TileLang				†
LayerNorm (BF16)	0.335	364	**0.270**	**124%**
RMSNorm (FP16)	2.52	294	**0.254**	990%†

RMSNorm $E_{\text{lib}} > 100\%$ because the fixed kernel fuses the scaling pass, reducing total memory traffic relative to PyTorch's unfused two-pass eager implementation; the fixed LayerNorm (124%) also slightly exceeds cuDNN's already-fused `F.layer_norm`.

C. Convolution: Partial Recovery via Implicit GEMM

a) Finding: Restructuring Conv2d as an implicit GEMM with FP16 Tensor Core acceleration, input padding to 16-byte alignment (addressing RC1), and an expanded autotune configuration space (addressing RC2) holds 36–76% of PyTorch/cuDNN efficiency across the 1×1–7×7 filter family on the A100-SXM4 (all correct), where cuDNN is substantially faster—the gap is narrowed but not closed.: The implicit GEMM restructuring unfolds the convolution input via on-the-fly tile formation (following the NHWC implicit GEMM approach [18]), converting the strided spatial access pattern into a dense matrix multiplication that Tensor Core units can accelerate in FP16. Input padding to a 16-byte boundary enables the compiler to emit `LDG.128` loads on the channel dimension, partially recovering the vectorization deficit (RC1). The expanded autotune space adds smaller `BLOCK_K` values (32, 16) for 3×3 filters and additional pipeline stage counts, covering configurations absent from the default list (RC2).

Of the residual ≈20% gap, Winograd selection contributes only ≈2–3% (measured: a 0.06% deterministic-vs-non-deterministic delta on the A100-SXM4 for 3×3 stride-1). The remaining ≈17–18% reflects general cuDNN implicit-GEMM tuning that the single-lowering Triton kernel does not match— kernel selection across data layouts, split-K, and shared-memory tile sizing (RC4). Closing this tuning gap, and adding Winograd support within a DSL kernel, are identified as future work.

D. GEMM and Reduction Kernels

Beyond the normalization kernels, we optimized the GEMM kernel and the full TileLang reduction/softmax family to test how broadly—and how completely—the RQ1 gaps recover.

Square matmul (Triton, FP16). Adding `@triton.autotune` with an expanded set of tile configurations and a `GROUP_SIZE_M` L2 cache swizzle (which reorders output tiles to improve L2 data reuse across the M-dimension) reduces latency from 1.08 ms to 0.79 ms (1.37×, $E_{\text{lib}} = 77\%$) on a 4096×4096 matmul, re-timed on the A100-SXM4; the same expanded search recovers E_{lib} from 32.5% to 81.8% (2.52×) at the RQ1 16384^2 shape.

TABLE IX
Convolution mitigation across the filter family on the A100-SXM4 (optimized implicit-GEMM Triton, FP16; input $32 \times 256 \times 128 \times 128$). E_{lib} = library efficiency relative to PyTorch/cuDNN; the mitigation holds 36–76% across the 1×1–7×7 filter family, all correct.

Filter	Baseline E_{lib}	Optimized E_{lib}
1×1	28.5%	**75.5%**
3×3	19.3%	42.4%
5×5	13.7%	38.5%
7×7	9.8%	36.1%

Measured under locked clocks (graphics/memory 1215 MHz); locked-clock re-timing over 100 runs shows 0.0–0.9% run-to-run relative std-dev on near-parity kernels, superseding the prior cross-session clock-drift caveat (which reflected unlocked clocks).

On the data-center A100, cuBLAS is fast enough that this gap narrows but does not close. Iterations 2–6 (persistent kernel, expanded configs, `max_num_imprecise_acc`) converged without further gain, confirming that the expanded configuration set and L2 swizzle are the effective ceiling for this shape. This confirms RC2 as a contributor to the GEMM gap and demonstrates that search-space expansion substantially narrows it for square shapes. This reconciles with the earlier default-grid finding that the stock 12-config autotune grid recovers $\Delta \approx 0$pp: the gain here is a property of the *expanded* search space (sweeping `GROUP_SIZE_M`, `num_warps`, and `num_stages`), not of a different problem shape.

Argmax (TileLang, FP16). Replacing global-memory element accesses with tiled shared-memory bulk loads and native FP16 I/O reduces latency from 11.97 ms to 2.31 ms ($5.2\times$) on a 8192×32768 reduction, re-timed on the A100-SXM4. The optimized kernel reaches $E_{\text{lib}} = 27\%$: it removes most of the unoptimized-TileLang overhead, but the A100's native `torch.argmax` (0.62 ms) remains $3.7\times$ faster, so parity is not reached on this GPU. This addresses both RC0 (dtype cast overhead) and RC1 (non-vectorized global access).

Reduction and softmax family (TileLang). The same RC0 fix—replacing `T.serial` reduction loops with native `T.reduce` and streaming the reduction dimension in tiles (an online, max-rescaled pass for the softmax variants) rather than materializing the full row in a fragment—generalizes across the *entire* TileLang reduction and normalization family on the A100-SXM4 (table X), not just a handful of kernels. Every catastrophically slow kernel recovers to PyTorch-comparable or better: `max_reduction`, `mean_reduction`, and `logsumexp` *exceed* PyTorch (132%, 99%, 582%), and `softmax` and `log_softmax` reach 86% and 71%—all numerically correct, with 4–29$\times$ speedups over the idiomatic kernels. This is the central evidence that the large normalization and reduction gaps in RQ1 are *authoring artifacts rather than DSL ceilings*: the idiomatic `T.serial` kernel is up to $300\times$ slower, while on the A100 the one-pattern `T.reduce` rewrite matches or beats the vendor library on every kernel in the family. The lone within-A100 exception is index-returning `argmax` (27%), where the index bookkeeping is intrinsically heavier and `torch.argmax` is already well tuned—note that the *value-only* reduction over the identical shape (`max_reduction`) reaches 132%, isolating the residual to the index pass. Re-timing the same optimized kernels on the GH200 confirms the pattern transfers—LayerNorm 120%, RMSNorm 1303%, MeanReduction 97%, MaxReduction 84%, Softmax 87%, LogSoftmax 90% (tables VII and X)—with one instructive exception: `logsumexp`, whose A100-tuned wide fp32 fragment register-spills on `sm_90` (255 registers, $\sim$280 GB of local-memory spill traffic, 12% occupancy by Nsight Compute, `ncu_summary.csv`), collapsing to 1.0%. The spill is an RC3-class effect, not the RC0 reduction idiom: retuning the fragment width recovers it only to 63%, so on the GH200 `logsumexp` is a genuine residual. That an "optimized" kernel validated on one GPU can be $100\times$ off on another, with no correctness signal, is precisely the evaluation gap this paper argues a single-target benchmark cannot close.

E. Summary and Remaining Gap

Table X summarizes the per-category mitigation results, re-timed on the A100-SXM4, with optimized-kernel efficiency also reported for the GH200. The outcomes separate cleanly into two kinds, and the distinction is the central result of RQ3. The TileLang *normalization and reduction family* are *authoring artifacts*: correcting RC0 (`T.serial`→`T.reduce` plus native-dtype, tiled streaming I/O) brings every kernel to PyTorch-comparable or better—LayerNorm 124%, RMSNorm 990%, and the full reduction/softmax family at 71–582% (table X)—closing gaps that were as large as $300\times$ in RQ1. These are not DSL ceilings: once the kernel is written the competent way, the DSL matches or beats the vendor library on the whole family on the A100, and on the GH200 for every family kernel but `logsumexp`—whose optimized config register-spills on `sm_90` (table X), a reminder that a fixed kernel still carries a hardware-specific tuning that must be re-validated per target. The remaining gaps are *genuine residuals* that survive best-effort optimization: Conv2d (42%), large square GEMM (Matmul 77% vs cuBLAS), and index-returning Argmax (27%)—here even the fully-optimized DSL kernel does not reach the A100's vendor library, because on the data-center A100 cuBLAS, cuDNN, and `torch.argmax` are themselves much faster and well-tuned. Of the residual Conv2d gap, only $\approx$2–3% is attributable to absent Winograd selection (RC4); the rest reflects general cuDNN implicit-GEMM tuning (kernel selection across data layouts, split-K, and shared-mem tile sizing) that the Triton kernel does not match.

a) Cross-architecture portability is itself an authoring concern.: Two kernels show that a *correct, hand-optimized* DSL kernel can pass on one GPU yet collapse on another, so single-GPU or single-shape benchmarking cannot certify it. `logsumexp` is one direction—fast on the A100 but register-spilling on the GH200 (table X); the in-tree TileLang Softmax is the other, and its cause is purely an authoring choice. That kernel caches the entire row in shared memory, so its per-block footprint grows with the row width ($2N$ bytes at fp16); on the A100 this monotonically erodes occupancy—the kernel

TABLE X
Summary of mitigation results. A100-SXM4 latencies in ms (lower is better); E_{lib} = optimized-kernel efficiency relative to PyTorch/cuDNN on each GPU. **Bold** = $\geq 95\%$ library efficiency on that GPU.

	A100-SXM4 (ms)			E_{lib}		
Kernel	PyTorch	Before	After	A100	GH200	Root cause
Triton						
Matmul	0.607	1.081	0.788	77.0%	68%	RC2
Conv2d	3.44	17.86	8.12	42.4%	34%	RC1+RC2
TileLang						
LayerNorm	0.335	364	**0.270**	**124%**	**120%**	RC0
RMSNorm	2.52	294	**0.254**	990%†	1303%†	RC0
Softmax	0.539	2.86	0.626	86.1%	87%	RC0
LogSoftmax	0.442	2.65	0.624	70.8%	90%	RC0
MaxReduction	0.560	11.97	**0.418**	**131.6%**	84%	RC0
MeanReduction	0.768	10.65	**0.766**	**99.5%**	**97%**	RC0
LogSumExp	2.40	4.18	**0.412**	**581.7%**	1.0%‡	RC0/RC3
Argmax	0.622	11.97	2.306	27.0%	22%	RC0+RC1

† RMSNorm $E_{\text{lib}} > 100\%$ due to kernel fusion; see table VIII. ‡ LogSumExp is the one family kernel whose optimized config does *not* transfer to the GH200: its A100-tuned wide fp32 fragment register-spills on sm_90 (255 regs, $\sim$280 GB local-memory spill traffic, 12% occupancy; GH200 `ncu_summary.csv`) $\rightarrow$ 1.0%; even retuned to the GH200-optimal fragment width it caps at 63%, an RC3-class residual rather than an authoring artifact. On the A100 (sm_80) the same kernel does *not* spill (0 local-memory traffic, 93 regs, 30.5% occupancy; A100 `ncu_summary.csv`), confirming the 581.7% is genuine and the spill is sm_90-specific. A100 *ms* columns are re-timed on the A100-SXM4 (locked clocks); the memory-latency-bound norm/reduction *Before* kernels are clock-invariant and match the unlocked profile. GH200 E_{lib} is the optimized-kernel efficiency from table VII (unlocked). *Before* is the unoptimized DSL kernel (Matmul: plain Triton; Conv2d: baseline vs. implicit-GEMM Triton at 3×3, cf. table IX); Matmul at 4096^2.

is $20\times$ slower than PyTorch at N=8192 (16 KB of shared memory) and $111\times$ slower at N=32768 (64 KB, past the 48 KB static budget), while staying numerically correct at every size (`softmax_authoring.csv`). The identical kernel runs at PyTorch parity on the GH200, whose larger shared-memory budget absorbs the allocation, so the defect is invisible to a GH200 or small-N benchmark even though the kernel is two orders of magnitude slow on the A100 at scale. The streaming variant—tiling the reduction over fixed 4 KB blocks instead of caching the whole row—holds parity across the sweep (ρ=0.55, table VII). "Stream the reduction; do not cache the full row" is thus a portable authoring pattern, and the collapse is a concrete instance of the evaluation gap whose root cause is authoring rather than the language or the hardware.

The normalization, reduction, and convolution mitigations in table X are re-timed on the A100-SXM4 (locked clocks); the matmul figure is the A100-SXM4 autotune value at 4096^2, reconciled against the 16384^2 result in §7.3. The optimized kernels were revalidated against the same ViperBench tolerances as the unmitigated kernels (5/5 pass): the core mechanism (`T.serial`$\rightarrow$`T.reduce` plus native-dtype I/O) reproduces correctly, and the edge-case correctness suite (NaN/Inf/denormal/all-equal) shows 0 crashes.

The normalization recoveries, while numerically dramatic, reflect correction of a known compiler deficiency (incorrect use of `T.serial` instead of `T.reduce`) rather than a novel optimization technique. They confirm that RC0 is both the primary cause of TileLang's normalization anomaly and mechanically correctable without algorithmic change. The convolution result (42% E_{lib} at 3×3 after RC1+RC2, 36–76% across the filter family) establishes a tighter upper bound on what can be achieved without Winograd support on the A100, and leaves $\approx$17–18pp of E_{lib} attributable to general cuDNN implicit-GEMM tuning that DSL kernels do not match (kernel selection across layouts, split-K, shared-mem tiling), and $\approx$2–3pp to absent Winograd selection (RC4); both are candidates for future compiler work, with RC2b-style search-space expansion the higher-leverage priority.

VIII. Discussion

The empirical results suggest that current GPU DSLs are not uniformly competitive with vendor libraries; rather, their effectiveness depends strongly on the operator class and the optimizations available in the compiler stack. This has two implications. First, the observed gaps identify concrete weaknesses in present DSL systems, particularly for convolution. Second, the results provide practical guidance for users deciding when DSL kernels are likely to be a good substitute for library calls. We discuss these implications for DSL developers and practitioners, and then outline the main limitations of the study.

A. Implications for DSL Developers

Our results point to three practical directions for improving current GPU DSL systems.

Improve vectorization for convolution accesses (requires compiler change). RC1 (section VI) shows that convolution performance is limited in part by the failure to emit wide `LDG.128` loads for strided spatial accesses. A direct compiler remedy is to extend alias and layout analysis to recognize the convolution access pattern and emit vectorized loads when the contiguous dimension is aligned (e.g., multiples of 8 for FP16). This change targets code generation rather than the programming model, and therefore does not require changes to the user-facing API.

Make auto-tuning shape-aware (partly user-space fixable today; broader search requires compiler change). RC2 (section VI) indicates that static tuning configurations transfer poorly from GEMM-like workloads to convolution. The RC2a portion is fixable in user space today: manually expanding the configuration list (e.g., `GROUP_SIZE_M`/`num_warps`/`num_stages`) recovers most of the GEMM gap, as shown in section VII. Closing the residual RC2b gap calls for auto-tuning strategies that are more adaptive to operator shape and memory-access structure. Prior work such as Ansor [19] shows that sketch-based search can discover effective schedules for irregular operators, including convolution, without relying on manually enumerated templates. A similar shape-aware search mechanism for Triton or TileLang would be a concrete step toward reducing this gap. Our own sweep illustrates both the hazard and its cost: tuning at a shape smaller than deployment can select a configuration *slower than the untuned default*, and tuning at the deployment shape—the obvious remedy—is expensive, because each candidate must

be timed on the full-size problem. Re-tuning a single slow kernel (attention) at its deployment shape took 211 s versus under a second at the small shape—a $\sim$1000$\times$ difference that compounds across kernels, dtypes, and shapes. Shape-representative tuning is therefore not free; this reinforces both that an exhaustively-tuned comprehensive benchmark is infeasible (section V) and that the lightweight comparability-plus-roofline heuristics of section VII are the more practical screen.

Support algorithmic alternatives for convolution (requires compiler change). RC4 highlights a limitation that cannot be addressed through local schedule tuning alone: current DSL implementations do not expose algorithmic choices comparable to those available in cuDNN. In particular, support for Winograd-style transformations would allow the compiler or runtime to choose among direct convolution, GEMM lowering, and Winograd-based execution depending on filter shape and problem size. Although this requires a larger system change than RC1 or RC2, and the wall-clock benefit attributable specifically to Winograd is small ($\approx$2–3%), it contributes to closing the remaining gap on common 3×3 convolutions.

The results also have immediate consequences for users who adopt DSLs to replace library-backed kernels.

Performance depends strongly on operator class. DSL kernels should not be assumed to match library performance by default. In our study, Triton is near parity on element-wise and normalization workloads, and can therefore offer a favorable trade-off between programmability and performance in these cases. For GEMM, the observed cost relative to cuBLAS may be acceptable when customization or fusion is required. Convolution is different: both Triton and TileLang remain substantially behind cuDNN in the evaluated settings. For convolution-heavy models, DSL kernels should therefore be treated as an optimization target that must be validated against a library baseline, not as a drop-in replacement.

The root-cause analysis can guide debugging. The mitigation results suggest a practical workflow for diagnosing underperforming DSL kernels. For TileLang reductions and normalization, developers should first replace serial `T.serial` accumulation with `T.reduce` (RC0a), since the dominant cost is exposed memory latency rather than thread synchronization; this is also where register pressure and spill should be inspected (RC3), which on our hardware appears in TileLang normalization rather than in convolution. For convolution, developers should check whether memory accesses are vectorized into wide `LDG.128` loads (RC1), then evaluate whether the tuning configuration space is too narrow for the target shape, distinguishing a config that is merely absent from the default grid (RC2a, fixable by expanding the search) from one that needs a broader search strategy (RC2b). For 3×3 filters, Winograd selection (RC4) is worth considering, but only as a small ($\approx$2–3%) contributor. This sequence provides a compact checklist for performance debugging in Triton and TileLang.

Benchmarks should gate on performance, not just correctness. The passes-but-slow result (section V-B) carries a concrete recommendation for benchmark designers: a correctness gate is necessary but not sufficient, and a kernel benchmark that reports speedup without gating on it certifies the wrong property. A lightweight, baseline-independent roofline check (section VII-A) is a practical acceptance criterion that needs no curated fast reference, and would have flagged every passes-but-slow kernel in our study.

B. Limitations

Our conclusions should be interpreted within the scope of the study.

Operator scope. We evaluate forward-pass kernels only. Backward kernels introduce different computation and memory patterns, including larger temporary state and more complex accumulation behavior, and may expose additional bottlenecks not captured here.

Hardware scope. Our primary measurements span two architectures, the NVIDIA A100-SXM4-40GB (Ampere, sm_80) and the NVIDIA GH200 (Grace Hopper, sm_90), with cross-architecture replay on the A100-PCIE-40GB (same sm_80) and H100-80GB-HBM3 confirming that the gaps generalize across form factors and GPU families. The results nonetheless characterize NVIDIA datacenter platforms rather than all accelerators; in particular, we do not evaluate ROCm or Intel XPU backends, although these are important targets for future work.

Software snapshot. Both Triton and TileLang are evolving systems. The measurements in this paper reflect the software versions listed in section III-G; later compiler or runtime changes may reduce some of the gaps we report. Our released benchmark suite is intended to support such re-evaluation.

IX. Related Work

This section situates our work in the literature on GPU kernel DSLs (section IX-A), performance analysis tools (section IX-B), and GPU benchmarking frameworks (section IX-C). Our study is the first to show that prevailing DSL and LLM-generated-kernel benchmarks admit performance-poor kernels, to characterize the resulting hidden gap with a hardware-grounded root-cause taxonomy, and to distill evaluation heuristics that flag poor kernels absent a comprehensive benchmark.

A. GPU Kernel DSLs

Halide [20] introduced the influential separation of algorithm specification from scheduling, enabling systematic search over loop transformations (tiling, vectorization, parallelism) without modifying the functional description. This principle informs both TVM [21] and TileLang [7].

Triton [3] departed from the explicit-schedule model in favor of an implicit tile abstraction in which the compiler infers shared memory layouts, synchronization, and pipelining. Its integration into PyTorch [5] has made it the most widely deployed GPU DSL. ThunderKittens [22] targets the opposite extreme: a thin C++ header library that exposes warp-level tile operations directly, achieving state-of-the-art attention throughput on H100 through explicit warp specialization. Our

study is complementary: we characterize performance at the DSL level rather than at the hand-optimized C++ level.

TVM [21] and its auto-scheduler Ansor [19] demonstrate that hierarchical program search substantially outperforms template-guided auto-tuning for irregular operator shapes, including convolution. This is consistent with RC2 in our analysis (section VI): the convolution search space is irregularly shaped relative to GEMM, and static configuration lists are insufficient. MLIR-based code generation [23], [24] offers a compiler-infrastructure approach to the same problem, generating near-peak-performance GEMM and fused-attention code through parametric Linalg transformations.

B. Performance Analysis and Profiling

TritonForge [13] proposes a profiling-guided LLM loop for automated Triton kernel optimization, using Nsight Compute metrics to identify bottlenecks. Their finding that memory coalescing failures and low occupancy account for the majority of kernel-level inefficiencies is consistent with our RC1 and RC3 findings. The difference is scope: TritonForge focuses on automation of the fix-generation step, while our study focuses on systematic characterization across a kernel taxonomy.

Empirical performance studies of GPU libraries have examined cuDNN [9], cuBLAS [8], and CUTLASS [11] extensively, but performance comparisons *between* DSL and library implementations at the scale and granularity of our study are absent from the literature.

C. Benchmarking GPU Software

The benchmarks closest to our setting evaluate DSL and LLM-generated kernels. TritonBench [14] evaluates Triton kernel generation by LLMs, measuring functional correctness and GPU efficiency as a fraction of hardware peak— a roofline-anchored metric that predates and motivates the anchor we adopt in section VII-A, and which we build on rather than introduce. KernelBench [12] is the de-facto benchmark for LLM kernel generation, but it gates on correctness alone and reports speedup only as an outcome; its single reward-hacking guard flags only implausibly fast kernels, a weakness automated generators have exploited. Both evaluate only a narrow slice of the (DSL, data type, shape, hardware) space. Our work differs in object and claim: we do not propose another benchmark, but show that the prevailing ones admit performance-poor kernels (section V), characterize the resulting hidden gap with hardware counters, and distill evaluation heuristics and optimization patterns that guide development absent a comprehensive benchmark.

MLPerf Inference [25] benchmarks end-to-end inference throughput but treats the computation stack as a black box. Our study operates at the kernel level to attribute performance differences to specific compiler behaviors.

X. Conclusion

We studied how to *evaluate* DSL GPU kernels: how a developer or an automated generator can tell whether a functionally correct Triton or TileLang kernel is actually performant. Across 22 kernels in five operator categories, we showed that the benchmarks practitioners rely on—KernelBench and TritonBench—gate on correctness and admit performance-poor kernels: a naive but idiomatic TileLang kernel passes the gate while running more than $300\times$ slower than PyTorch, and the gate's reward-hacking guard never fires because it only watches for kernels that are too *fast*. Because a benchmark comprehensive enough to close this gap is combinatorially infeasible, we instead characterized the hidden gap and distilled it into practical guidance.

The hidden gap is highly structured. Most of it is an *authoring* artifact: correcting a single TileLang idiom (`T.serial`→`T.reduce` with native-dtype I/O) recovers the normalization and reduction family to vendor-library parity or better—on both GPUs we test, with a single diagnosed exception (`logsumexp`, whose optimized kernel register-spills on the GH200)—closing gaps as large as $1500\times$ (the naive-to-optimized authoring cliff on the GH200). The remainder is a *genuine residual*—convolution and large GEMM—where even a best-effort DSL kernel trails cuDNN/cuBLAS; absent Winograd selection contributes only ≈2–3% of the convolution gap, the rest reflecting mature implicit-GEMM tuning. We separate causes by where the fix belongs (user-space authoring, compiler code generation, library maturity) and validate that lightweight heuristics—a comparability screen against the vendor baseline, anchored by a baseline-independent roofline fraction—reliably tell well-written kernels from poor ones, flagging the residuals (roofline ≤ 0.28) while clearing the recovered family (roofline 0.4–0.9).

Current GPU DSLs offer a strong programming model, but a correct kernel is not yet a fast one, and existing benchmarks do not close that gap. Until performance-aware benchmarks and compiler support mature, the evaluation heuristics and optimization patterns in this paper give developers—and the kernel-generating tools increasingly producing DSL code—a practical way to judge and improve kernel quality.

References

[1] M. Wolfe, "More iteration space tiling," in *Proceedings of the 1989 ACM/IEEE Conference on Supercomputing*, ser. Supercomputing '89. New York, NY, USA: Association for Computing Machinery, 1989, p. 655–664. [Online]. Available: https://doi.org/10.1145/76263.76337

[2] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, p. 65–76, Apr. 2009. [Online]. Available: https://doi.org/10.1145/1498765.1498785

[3] P. Tillet, H. T. Kung, and D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, ser. MAPL 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 10–19. [Online]. Available: https://doi.org/10.1145/3315508.3329973

[4] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.

[5] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher,

Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 929–947. [Online]. Available: https://doi.org/10.1145/3620665.3640366

[6] sebastienwood, "Example conv2d in triton," GitHub Discussion #591, https://github.com/triton-lang/triton/discussions/591, 2022, accessed: 2026-03-25.

[7] L. Wang, Y. Cheng, Y. Shi, Z. Mo, Z. Tang, W. Xie, T. Wu, L. Ma, Y. Xia, J. Xue, F. Yang, and Z. Yang, "Tilelang: Bridge programmability and performance in modern neural kernels," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=Jb1WkNSfUB

[8] NVIDIA Corporation, "cuBLAS: Basic linear algebra on nvidia gpus," https://docs.nvidia.com/cuda/cublas/, 2026.

[9] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," 2014. [Online]. Available: https://arxiv.org/abs/1410.0759

[10] ——, "cudnn: Efficient primitives for deep learning," 2014. [Online]. Available: https://arxiv.org/abs/1410.0759

[11] NVIDIA Corporation, "CUTLASS: Cuda templates and python dsls for high-performance linear algebra," https://github.com/NVIDIA/cutlass, 2017.

[12] A. Ouyang, S. Guo, S. Arora, A. L. Zhang, W. Hu, C. Ré, and A. Mirhoseini, "KernelBench: Can LLMs write efficient GPU kernels?" 2025. [Online]. Available: https://arxiv.org/abs/2502.10517

[13] H. Li, K. Man, P. Kanuparthy, H. Chen, W. Sun, S. Tallam, C. Zhu, K. Zhu, and Z. Qian, "Tritonforge: Profiling-guided framework for automated triton kernel optimization," 2025. [Online]. Available: https://arxiv.org/abs/2512.09196

[14] J. Li, S. Li, Z. Gao, Q. Shi, Y. Li, Z. Wang, J. Huang, H. Wang, J. Wang, X. Han, Z. Liu, and M. Sun, "TritonBench: Benchmarking large language model capabilities for generating triton operators," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 23 053–23 066. [Online]. Available: https://aclanthology.org/2025.findings-acl.1183/

[15] NVIDIA Corporation, "NVIDIA nsight compute," https://developer.nvidia.com/nsight-compute, 2024.

[16] S. Xie, S. Xie, D. Ran, W. Yang, and T. Xie, "AKO: Agentic kernel optimization," https://tongminglaic.github.io/AKO, 2026, technical report.

[17] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," 2023. [Online]. Available: https://arxiv.org/abs/2307.08691

[18] Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, M. Guo, and Y. Zhu, "Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*, 2021, pp. 214–225.

[19] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating High-Performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 863–879. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zheng

[20] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines," vol. 48, no. 6. New York, NY, USA: Association for Computing Machinery, Jun. 2013, p. 519–530. [Online]. Available: https://doi.org/10.1145/2499370.2462176

[21] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated End-to-End optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 578–594. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/chen

[22] B. F. Spector, S. Arora, A. Singhal, A. Parthasarathy, D. Y. Fu, and C. Re, "Thunderkittens: Simple, fast, and $\textit{Adorable}$ kernels," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=0fJfVOSUra

[23] N. Katel, V. Khandelwal, and U. Bondhugula, "Mlir-based code generation for gpu tensor cores," in *Proceedings of the 31st ACM SIGPLAN International Conference on Compiler Construction*, ser. CC 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 117–128. [Online]. Available: https://doi.org/10.1145/3497776.3517770

[24] N. Vasilache, O. Zinenko, A. J. C. Bik, M. Ravishankar, T. Raoux, A. Belyaev, M. Springer, T. Gysi, D. Caballero, S. Herhut, S. Laurenzo, and A. Cohen, "Composable and modular code generation in mlir: A structured and retargetable approach to tensor compiler construction," 2022. [Online]. Available: https://arxiv.org/abs/2202.03293

[25] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, G. Schmuelling, C.-J. Wu, B. Anderson, M. Breughe, M. Charlebois, W. Chou, R. Chukka, C. Coleman, S. Davis, P. Deng, G. Diamos, J. Duke, D. Fick, J. S. Gardner, I. Hubara, S. Idgunji, T. B. Jablin, J. Jiao, T. S. John, P. Kanwar, D. Lee, J. Liao, A. Lokhmotov, F. Massa, P. Meng, P. Micikevicius, C. Osborne, G. Pekhimenko, A. T. R. Rajan, D. Sequeira, A. Sirasao, F. Sun, H. Tang, M. Thomson, F. Wei, E. Wu, L. Xu, K. Yamada, B. Yu, G. Yuan, A. Zhong, P. Zhang, and Y. Zhou, "Mlperf inference benchmark," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 446–459.